



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

Aula – Arquiteturas Orientadas a Mensagens

Prof.: Filipe Nhimi

Agenda

- ❑ Introdução/Revisão
- ❑ Arquitetura Monolítica
- ❑ Arquitetura Distribuída
- ❑ Arquitetura Assíncrona
- ❑ Conclusão

O que é Arquitetura de Software?

“É a **organização** fundamental de um sistema, composta por **componentes** (módulos), seus **relacionamentos** e os princípios que governam seu projeto e evolução. Ela define a **estrutura geral** do software, como seus elementos interagem entre si e a visão de alto nível do sistema”

Roger Pressman



O que é Arquitetura de Software?

“Segundo **Mark Richards** e **Neal Ford**, em *Fundamentos da Arquitetura de Software*, a Arquitetura de Software é entendida como:

A **estrutura fundamental** de um sistema de software, incluindo os **componentes**, suas **responsabilidades**, as **relações entre esses** componentes e os princípios que orientam sua evolução.”



Mark Richards



Neal Ford

ARQUITETURA != APENAS ESTRUTURA ESTÁTICA

O que é Arquitetura de Software?

“É um conjunto de **decisões** difíceis de serem mudadas no futuro.”

Martin Fowler



O que é Arquitetura de Software?

“Segundo Bass, Clements e Kazman, Arquitetura de Software é a **estrutura ou estruturas de um sistema**, compostas por **elementos de software**, suas propriedades visíveis externamente e as **relações** entre esses elementos.”

ESTRUTURA != CÓDIGO

ELEMENTOS != CLASSES APENAS

RELAÇÕES != CHAMADAS DE MÉTODO SOMENTE



Len Bass



Paul Clements



Rick Kazman

Principais Desafios

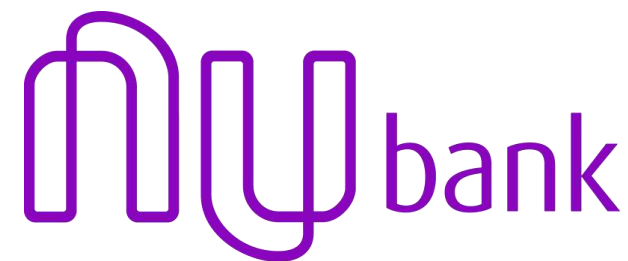
Técnicos e de Negócio

- Aplicações Legadas
- Infraestrutura
- Integrações
- Performance
- Escalabilidade
- Disponibilidade
- Confiabilidade
- Usabilidade
- Interoperabilidade

- Segurança
- Observabilidade
- Modelo de negócio
- Legislação
- Inovação
- Necessidades dos clientes
- Custos
- Prazos
- Comprometimento dos *stakeholders*

Softwares de qualquer porte vão apresentar **problemas** como **consequência** de **decisões ruins** tomadas em sua modelagem arquitetural.

Prof. Filipe Nhimi



Uber

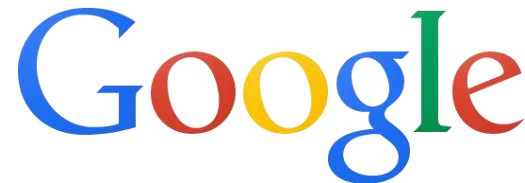


NETFLIX



mercado
livre

TODOS POSSÍVEIS
E MAIS UM POUCO!

ifood



ny bank



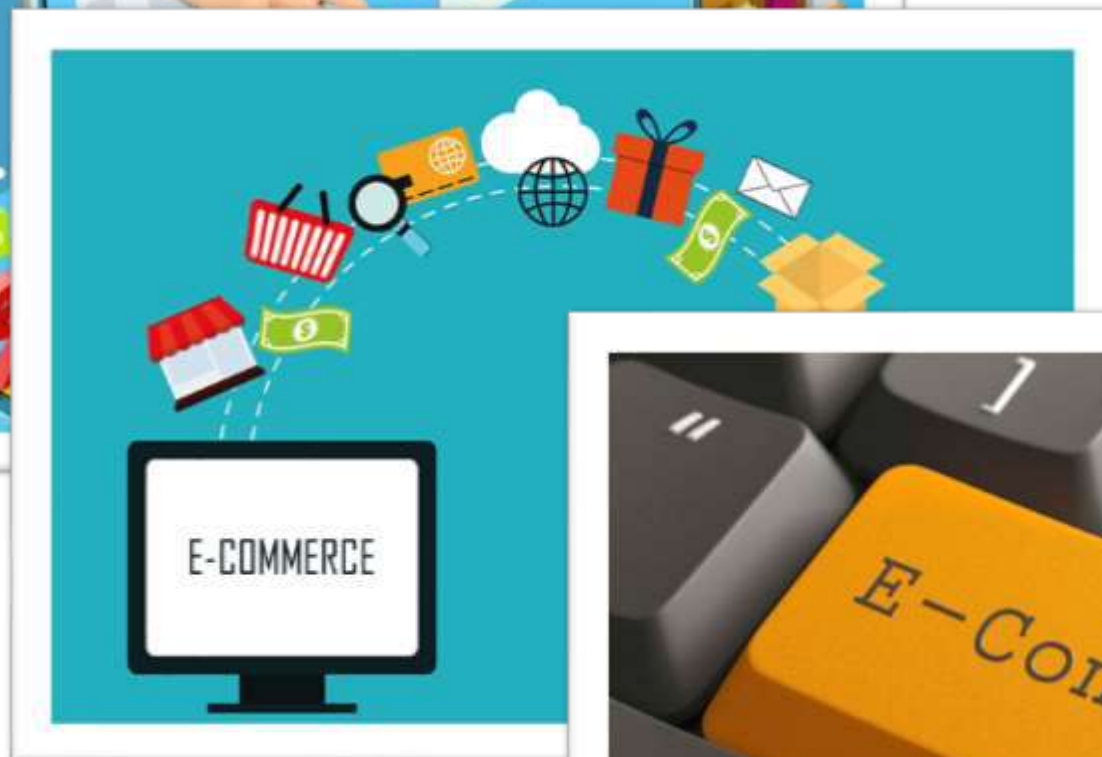
**mercado
livre**

NETFLIX

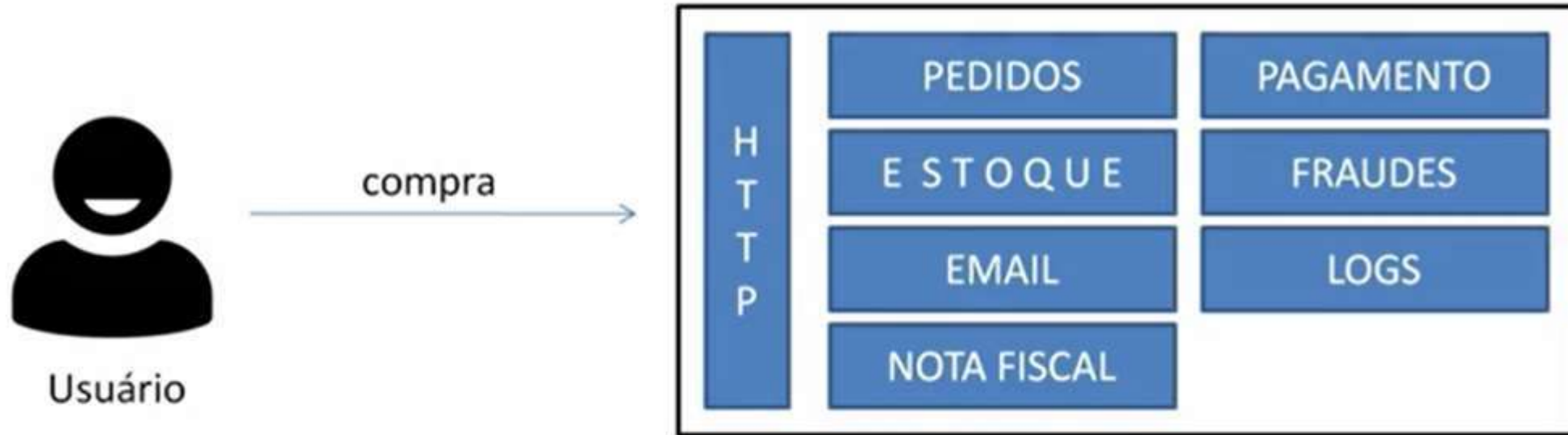
O que há de mais comum
nessas aplicações?



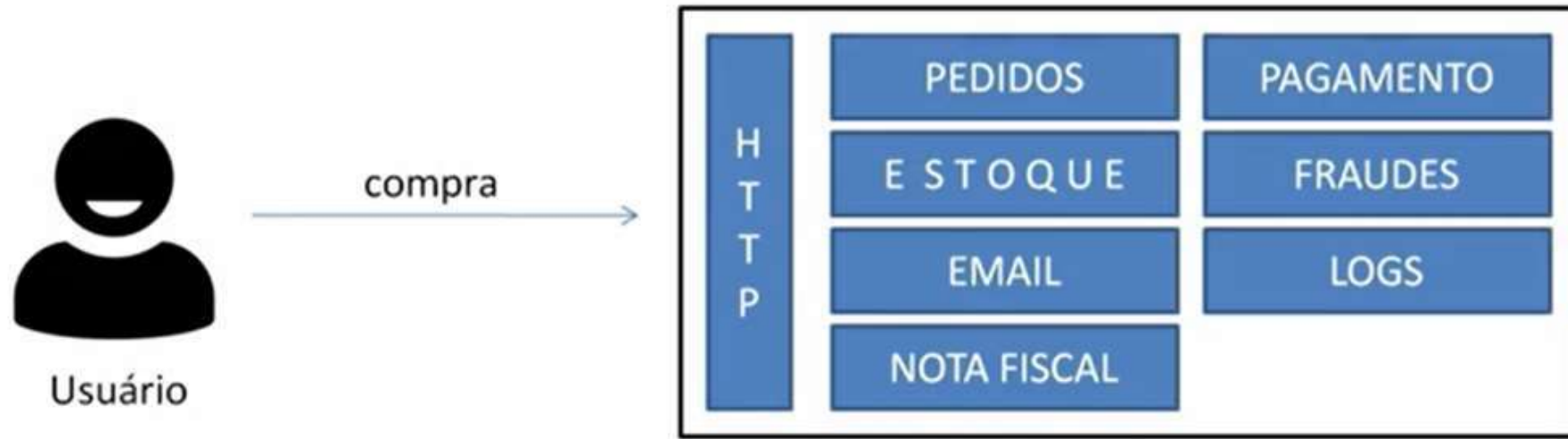
Alta Complexidade
Alta Volumetria



E-commerce XPTO com Arquitetura Monolítica



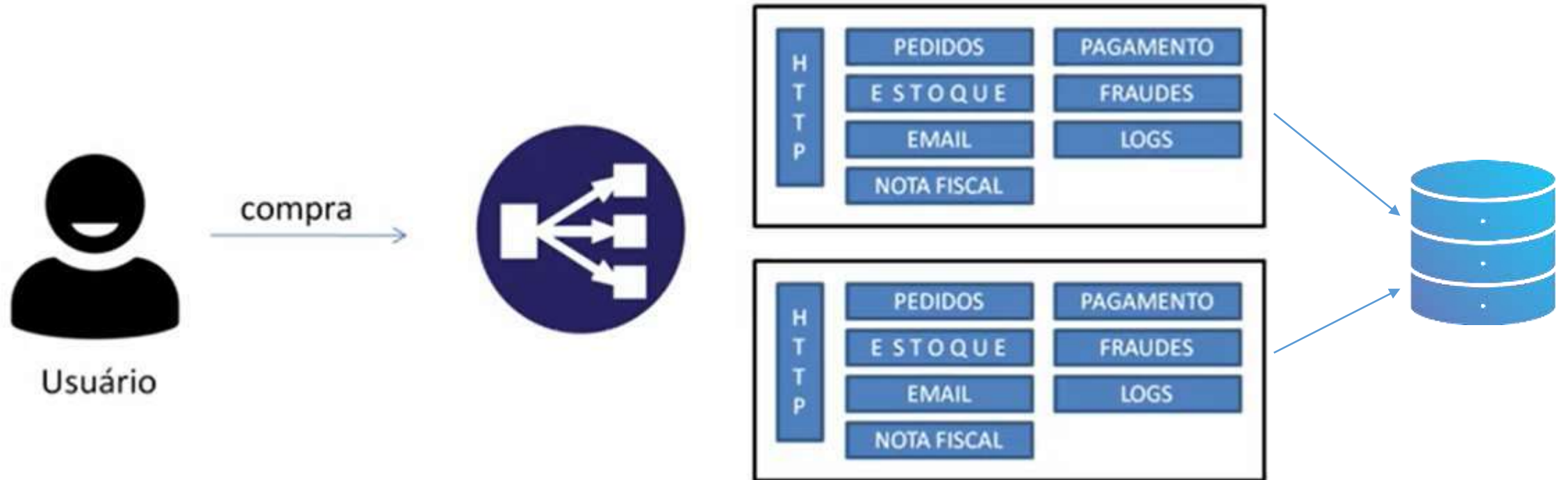
E-commerce XPTO com Arquitetura Monolítica



- **Escalabilidade limitada** — é preciso escalar tudo junto.
- **Deploy crítico** — uma mudança derruba o sistema inteiro.
- **Baixa manutenibilidade** — código grande, acoplado e complexo.
- **Performance instável** — um módulo lento afeta todos.
- **Falha única** — qualquer erro interrompe todo o e-commerce.
- **Dificuldade de inovação** — preso a uma linguagem/tecnologia.
- **Time lento** — muitos devs mexendo no mesmo código.
- **Banco único como gargalo** — concorrência, deadlocks, lentidão.

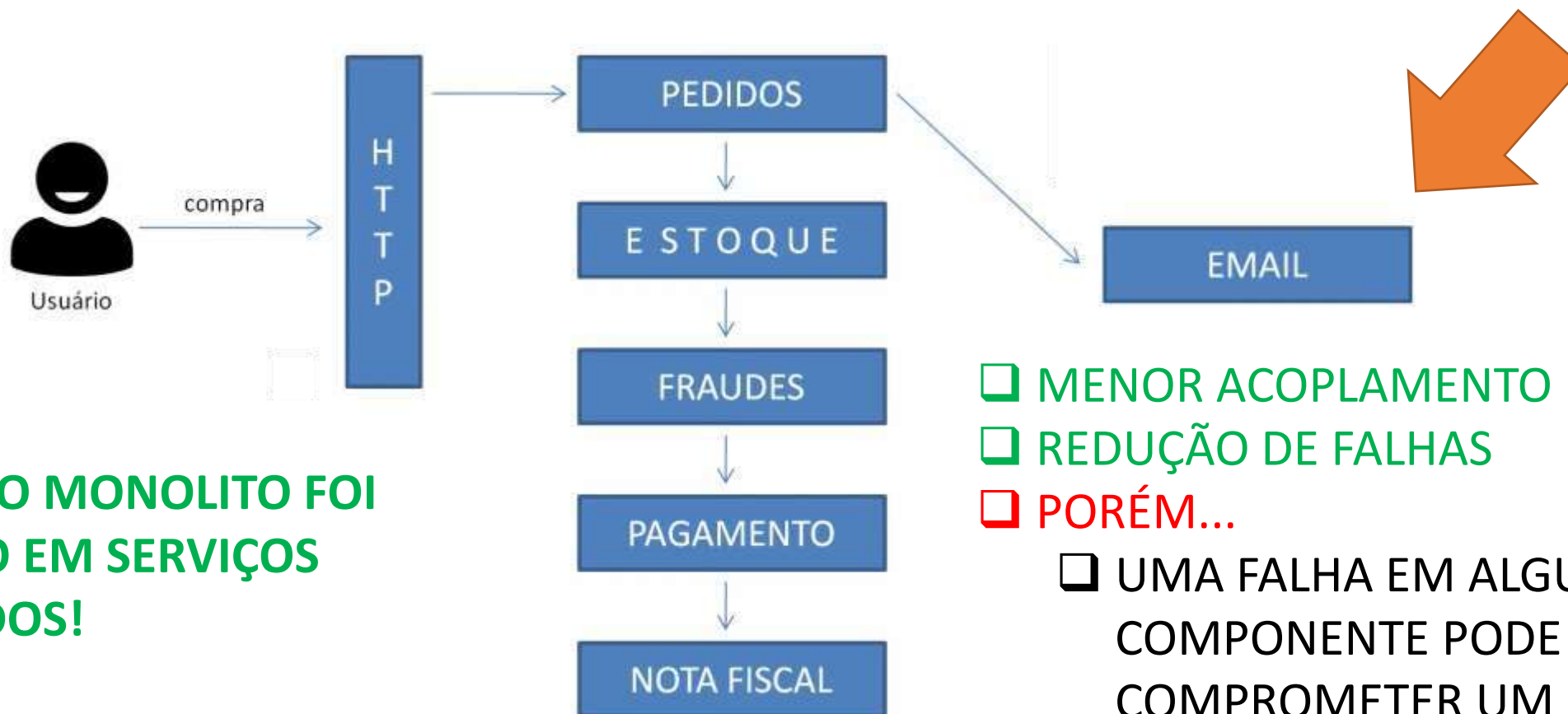
E-commerce XPTO

Escalonamento com réplicas e um Load Balancer



E-commerce XPTO

Com Arquitetura Distribuída



NOTE QUE O MONOLITO FOI QUEBRADO EM SERVIÇOS DISTRIBUÍDOS!

- ☐ MENOR ACOPLAMENTO
- ☐ REDUÇÃO DE FALHAS
- ☐ PORÉM...
 - ☐ UMA FALHA EM ALGUM COMPONENTE PODE COMPROMETER UM FLUXO IMPORTANTE DE NEGÓCIO!

E-commerce XPTO

Com Arquitetura Distribuída

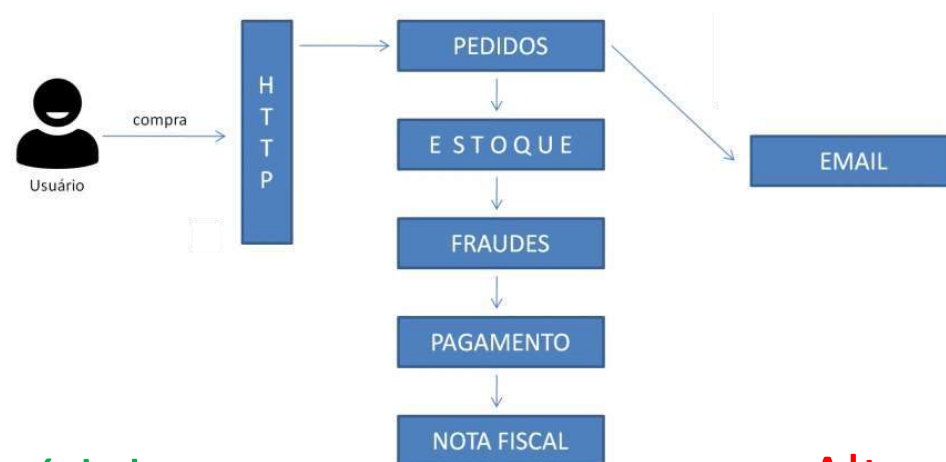


MELHOROU POIS AGORA
CONSIGO **ESCALAR SERVIÇOS**
DE FORMA INDIVIDUALIZADA

**PORÉM, A APLICAÇÃO NÃO É
RESILIENTE, PODE SOFRER COM
LATÊNCIA E AINDA FUNCIONA
EM COMUNICAÇÃO SÍNCRONA**

E-commerce XPTO

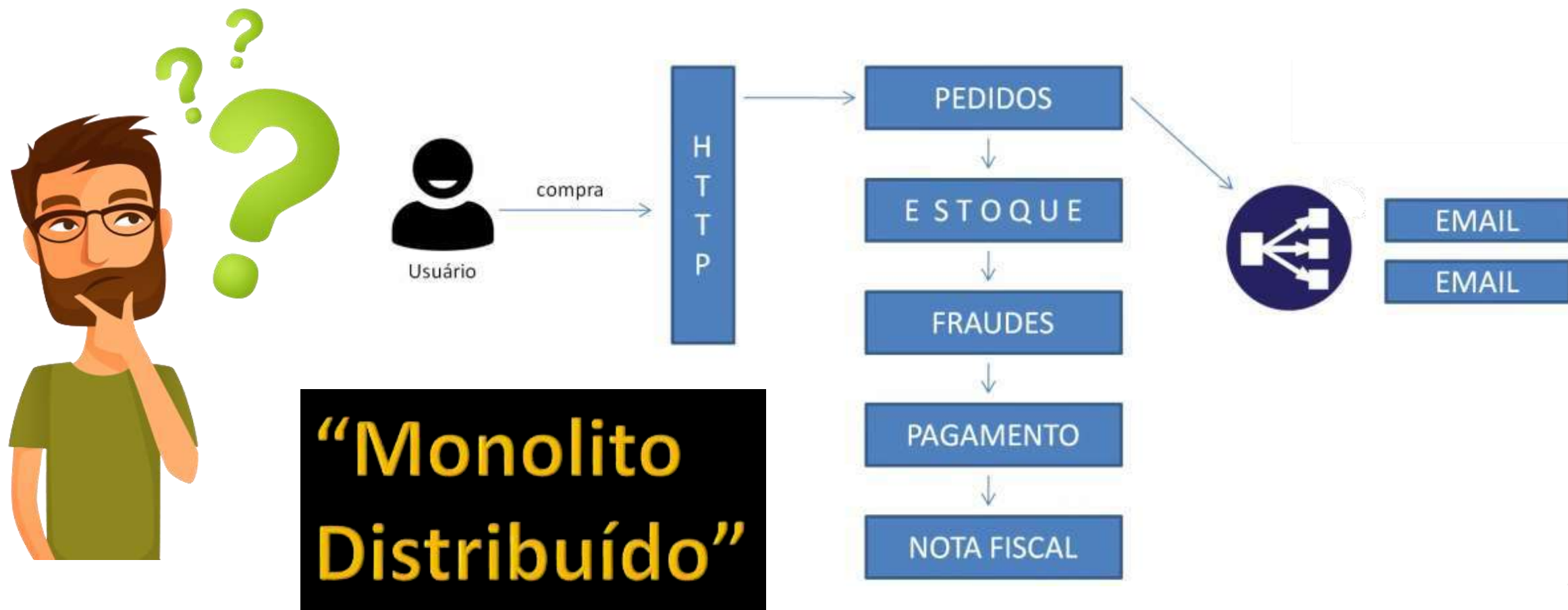
Com Arquitetura Distribuída



“Monolito Distribuído”

- Escalabilidade por módulo
- Deploy independente
- Isolamento parcial de falhas
- Manutenção mais simples
- Observabilidade mais clara
- Domínios separados

- Alto acoplamento temporal via HTTP
- Cascata de latência e falhas
- Orquestração rígida
- Baixa resiliência
- Não absorve picos



Meu negócio tem perdido vendas em função de erros quando tem sobrecarga na aplicação. Vejo pela plataforma de monitoramento que as vezes alguns serviço falham e todo o fluxo de vendas é comprometido. Temos relatos de lentidão em alguns momentos, e-mails que não são entregues e pedidos processados parcialmente sem a correta emissão da nota fiscal.



Preciso que minha arquitetura seja confiável!

Está tudo muito acoplado, tem falhas eventuais e principalmente sob carga.

Não posso continuar perdendo vendas. Está comprometendo o negócio e a credibilidade da plataforma.

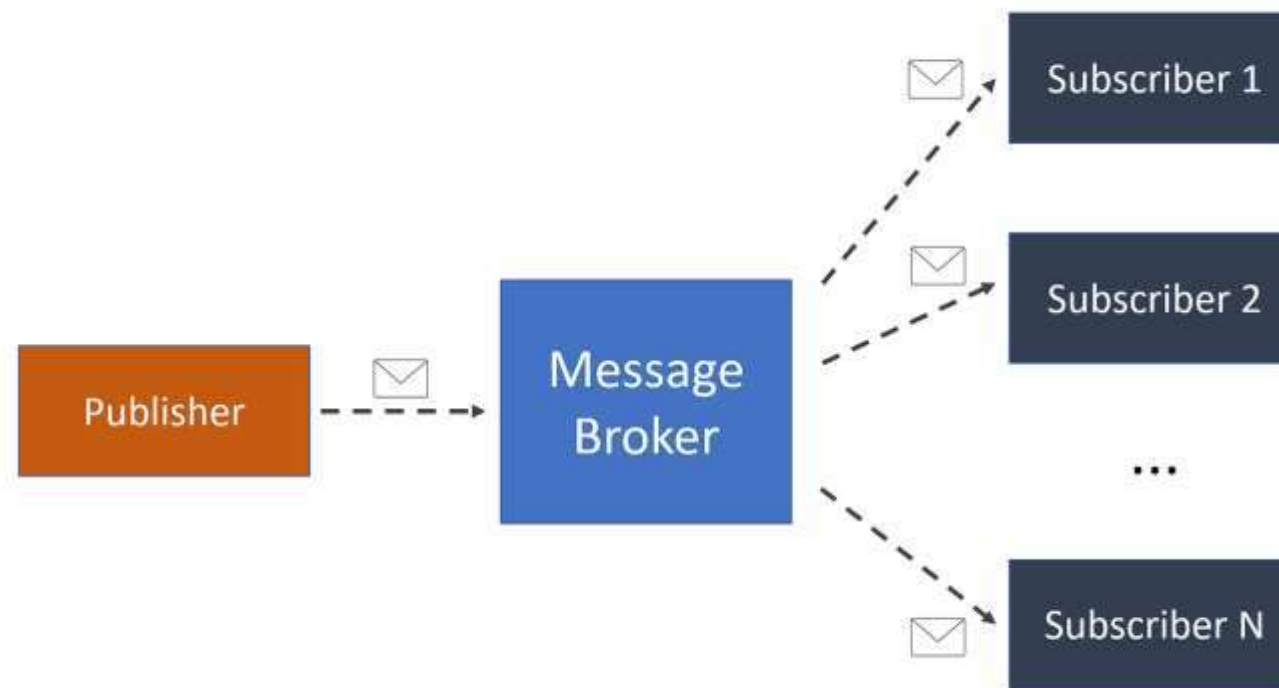
Decidimos reescrever a solução. Dado o cenário, o que podemos usar em termos de Arquitetura de Software?

Middleware Orientado a Mensagens (MOM)

Middleware Orientado a Mensagens (*Message-Oriented Middleware – MOM*) é uma infraestrutura tecnológica que permite a aplicação do **estilo arquitetural orientado a mensagens** em que sistemas comunicam entre si por meio de troca de **mensagens assíncronas**, utilizando mecanismos como filas, tópicos ou streams, implementados por um **broker** de mensageria, para desacoplar emissores e consumidores.

Middleware Orientado a Mensagens (MOM)

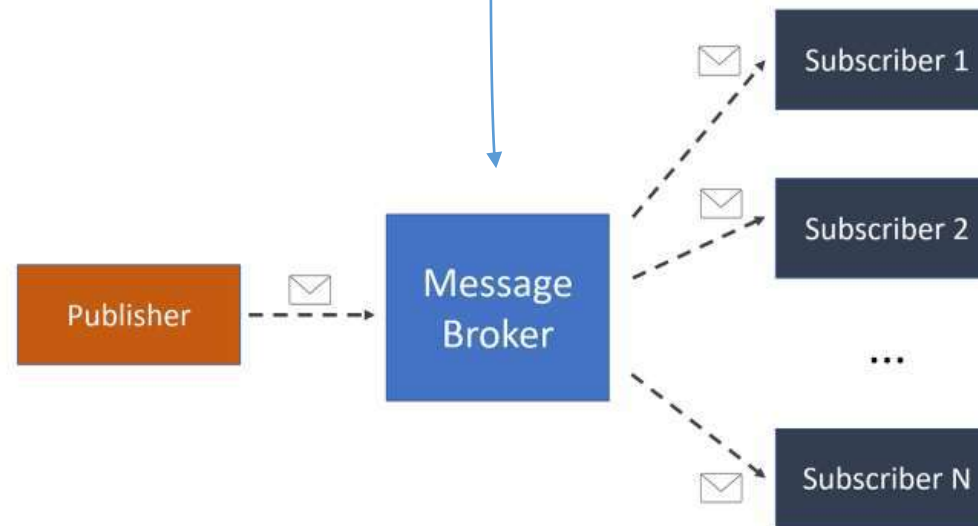
O MOM atua como uma **camada intermediária**



Middleware Orientado a Mensagens (MOM)

Infraestrutura

Componente dessa infraestrutura



Mecanismos de Comunicação

Queue, Topic, Stream

Middleware Orientado a Mensagens (MOM)

Estilos Arquiteturais Relacionados que utilizam estruturas similares a um MON

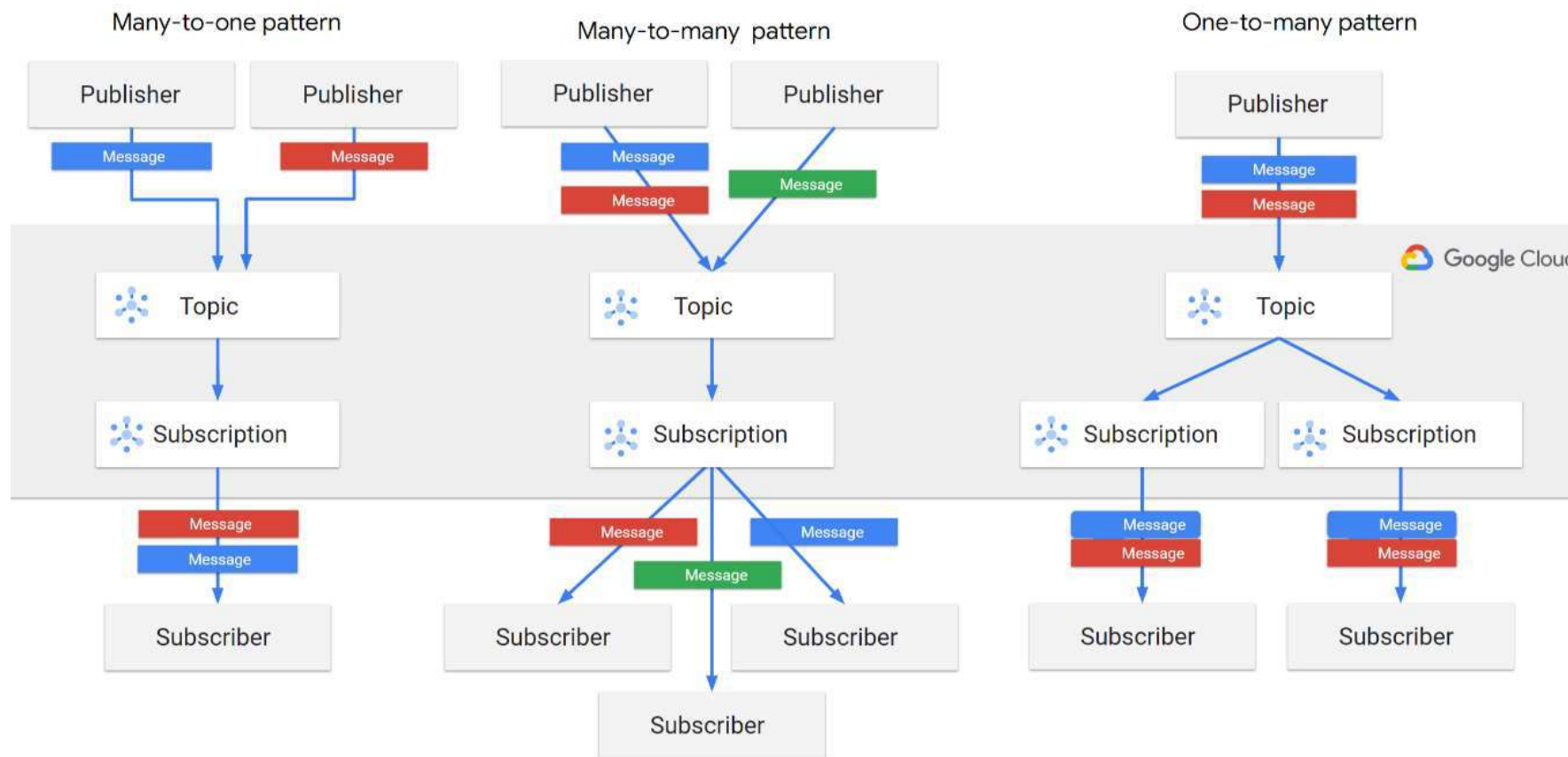
Point-to-Point (Fila):

- Um produtor envia mensagens para uma **fila**.
- Permite múltiplos consumidores concorrentes
- Um único consumidor as processa.
- O consumidor remove a mensagem da fila
- Modelo FIFO (geralmente)
- Padrão mais usado em **tarefas assíncronas de alta carga**.
- Garantia de entrega (“at least once”, “at most once”, etc.)
- Acoplamento fraco entre produtor e consumidor
- Backpressure natural (fila cresce quando o consumidor está lento)



Middleware Orientado a Mensagens (MOM)

Estilos Arquiteturais Relacionados que utilizam estruturas similares a um MON

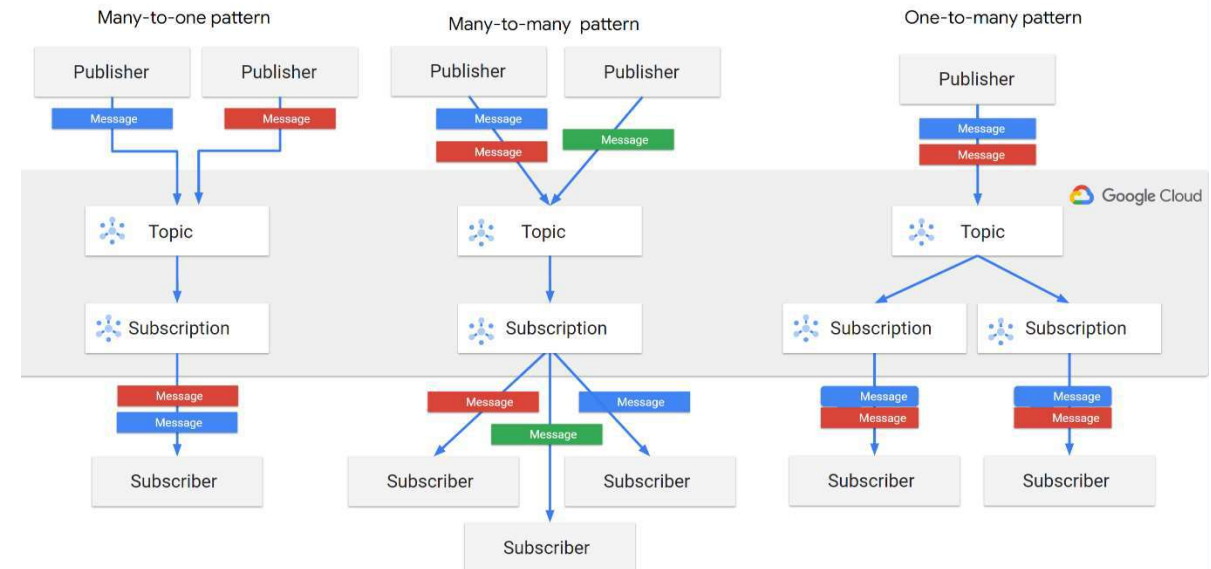


Middleware Orientado a Mensagens (MOM)

Estilos Arquiteturais Relacionados que utilizam estruturas similares a um MON

Publish/Subscribe (Tópico):

- Produtor publica em um **tópico**.
- Todos os assinantes interessados recebem.
- Base para sistemas **event-driven** e **broadcast**.
- Comunicação 1→N
- Baixo acoplamento
- Ideal para integrações e reações a eventos

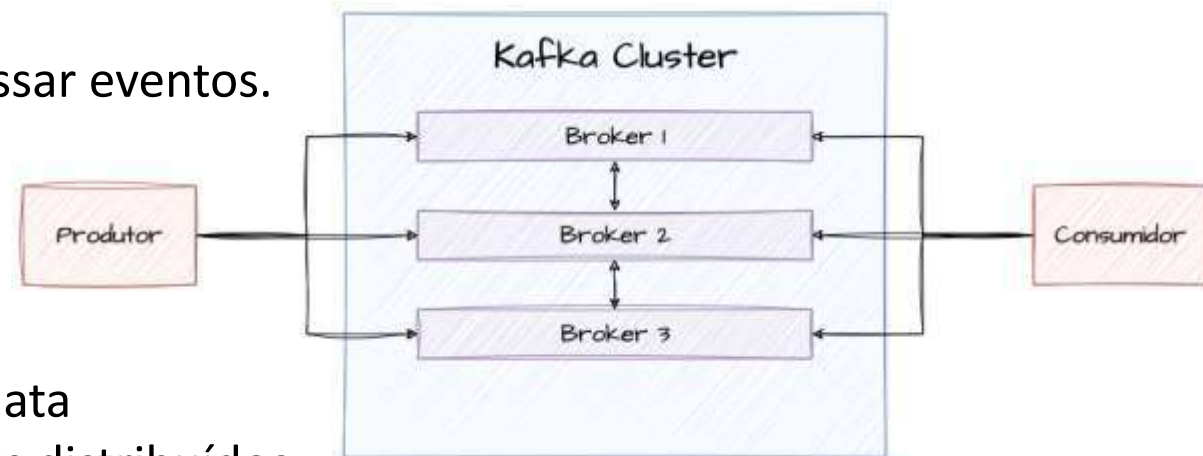


Middleware Orientado a Mensagens (MOM)

Estilos Arquiteturais Relacionados que utilizam estruturas similares a um MON

Event Streaming:

- Mensagens não são removidas.
- Consumidores independentes podem reprocessar eventos.
- Reprocessamento de eventos
- Ordenação
- Alto *throughput*
- Persistência longa
- Fundamentais em arquiteturas reativas e big data
- Soluções como **Kafka**, **Pulsar**, **Kinesis** usam logs distribuídos imutáveis.



Middleware Orientado a Mensagens (MOM)



Google Cloud Pub/Sub



Azure Service Bus



Voltamos para o e-commerce XPTO com **Arquitetura Monolítica**



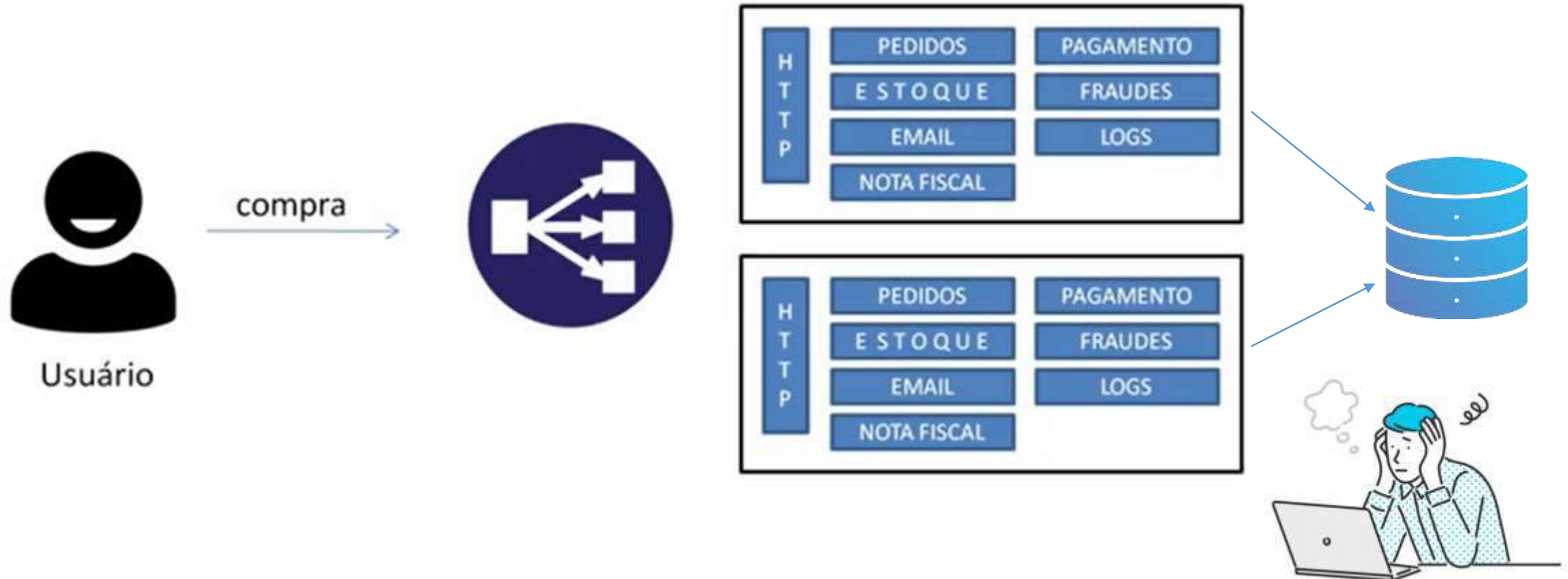
Usuário

compra



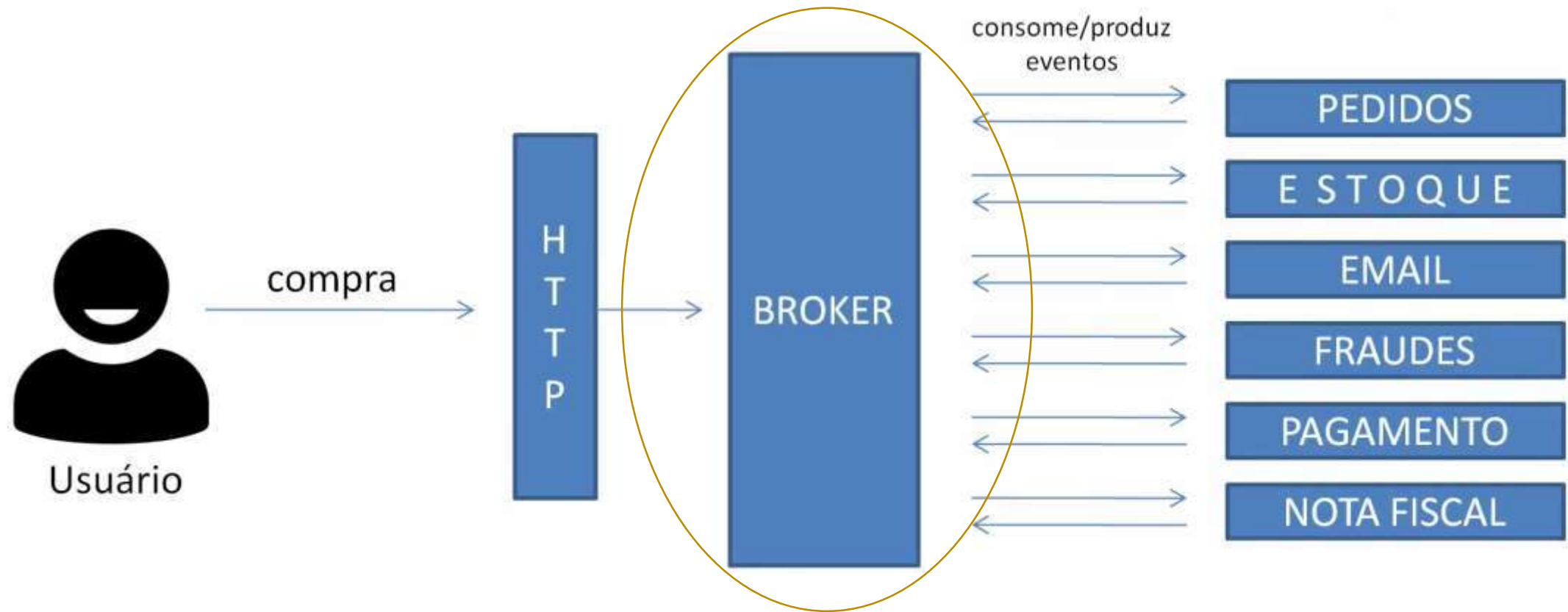
Voltamos para o e-commerce XPTO

com **Arquitetura Distribuída, “monolito fatiado”**



E-commerce XPTO

Com **Arquitetura Assíncrona** utilizando MOM



Message Broker



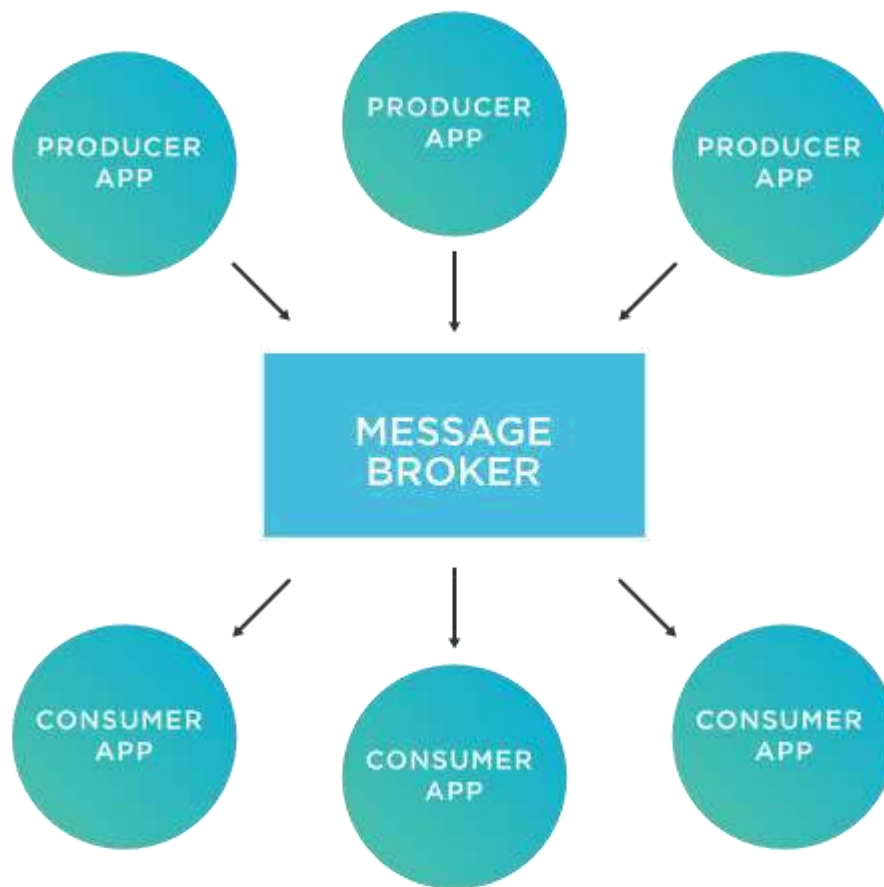
Recebe encomendas e cartas de remetentes, **armazena**,
monta **rotas** para **entrega** ou **retirada** pelos destinatários

Message Broker

Recebe
mensagens
de produtores

Armazena e
persiste mensagens

Entrega mensagens
aos consumidores



Consumidores podem
buscar mensagens

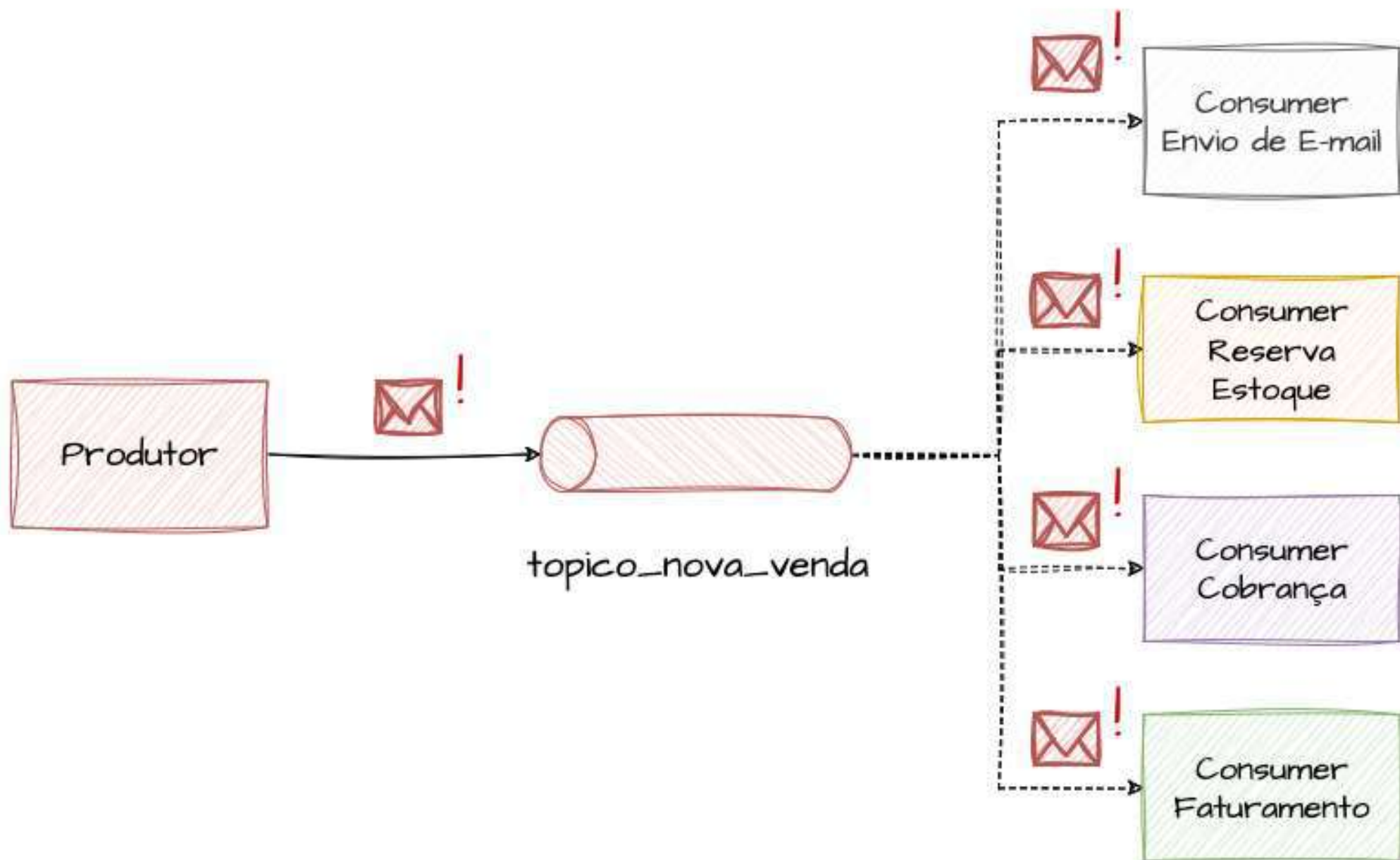
Roteia mensagens

Absorve
picos de carga

Isola falhas
entre serviços

Permite
reprocessamento (streams)

Implementa
mecanismos avançados
(retry, backoff, DLQ, idempotência,
ordenação, checkpoints, offset tracking)



Conclusão

Quando usar MOM?

- Desacoplamento entre componentes
- Alta escalabilidade
- Comunicação assíncrona
- Tolerância a falhas e resiliência
- Processamento distribuído
- Filas para *workloads* pesados

A idéia é substituir integrações síncronas sensíveis

Conclusão



“No Silver Bullet”

Não existe tecnologia capaz de eliminar de forma mágica a complexidade inerente do software.



Fred Brooks

(1931 – 2022)

"in memoriam"

Conclusão

Portanto, **MOM não é uma solução mágica**, e sim **uma decisão arquitetural** que traz benefícios claros quando aplicada corretamente, mas que **também introduz novos desafios** como **monitoramento, idempotência, orquestração e governança de mensagens**.

Referências

1. Hohpe & Woolf — Enterprise Integration Patterns (MOM/EIP)

HOHPE, Gregor; WOOLF, Bobby. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston: Addison-Wesley, 2003.

2. Bass, Clements & Kazman — Arquitetura de Software

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. *Software Architecture in Practice*. 4. ed. Boston: Addison-Wesley, 2021.

3. Taylor, Medvidovic & Dashofy — Fundamentos de Arquitetura

TAYLOR, Richard; MEDVIDOVIC, Nenad; DASHOFY, Eric. *Software Architecture: Foundations, Theory, and Practice*. Hoboken: Wiley, 2009.

4. Coulouris et al. — Sistemas Distribuídos

COULOURIS, George; DOLLIMORE, Jean; KINDERG, Tim; BLAIR, Gordon. *Distributed Systems: Concepts and Design*. 5. ed. Boston: Addison-Wesley, 2011.

5. Fowler — Padrões de Aplicações Corporativas

FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2003.

6. Ford & Richards — Arquitetura Moderna

FORD, Neal; RICHARDS, Mark. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol: O'Reilly Media, 2020.

7. Kreps & Pathella — Event-driven Microservices / Streaming

KREPS, Jay; PATELLA, Adam. *Building Event-Driven Microservices: Leveraging Organizational Data at Scale*. Sebastopol: O'Reilly Media, 2023.

8. Fielding — Estilos Arquiteturais (REST e estilos de comunicação)

FIELDING, Roy Thomas. *Architectural Styles and the Design of Network-based Software Architectures*. 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000.

9. Brooks — No Silver Bullet

BROOKS, Frederick P. *No Silver Bullet: Essence and Accidents of Software Engineering*. *Computer*, v. 20, n. 4, p. 10-19, 1987.